BWC Adaptive Selector: Pair-Aware Runtime Prefetcher Selection in ChampSim

Zhaofeng Luo Kijun Shin

15-740 Computer Architecture

0. Abstract

Hardware prefetchers hide memory latency, but aggressive prefetch traffic can consume shared cache capacity, memory queues, and off-chip bandwidth. This is especially important in multicore systems, where a locally plausible prefetch stream can delay demand requests from a peer. We address this prefetch-control problem in ChampSim by building the BWC Adaptive Selector, a pair-aware L2 runtime selector over original SPP, an FDP-style SPP variant, and BOP. BWC observes local stream features such as page growth, line growth, small-delta ratio, and RFO share, then uses peer/shared signals to decide whether the local expert choice is safe under contention. OpenEvolve was used in the final phase to refine only the pair-coordination logic, leaving expert prefetcher internals fixed. The retained selector improves all four listed single-core workloads over SPP by 0.8569% to 2.6685% IPC and all ten reported two-core pairs by 0.0515% to 4.2142% weighted speedup. In the focused Phase 2 comparison, OpenEvolve fixes the `bzip2 + lbm` regression and reduces the `mcf + lbm` worst-case loss. The main insight is that prefetch control should be peer-aware: locally reasonable prefetch decisions can become globally harmful under shared-resource contention.

1. Introduction

Modern data prefetchers exploit regular memory behavior to fetch lines before the core demands them. This can substantially improve performance for streaming and structured workloads. However, prefetches are not free: each request occupies queues, MSHRs, cache space, and DRAM bandwidth. In multicore systems, these costs cross core boundaries. A prefetcher that helps one core locally can hurt another core through shared LLC and memory-system pressure.

The project began from a bandwidth-aware control question: can a runtime policy control an SPP-like prefetcher better than static or accuracy-only throttling? Accuracy is useful but incomplete. A low-accuracy stream can still provide coverage or memory-level effects, while a confident predictor can still create harmful traffic. We therefore treat prefetching as a control problem, not only an address-prediction problem.

The final artifact is the BWC Adaptive Selector. Rather than inventing a new predictor, it chooses among experts with different behaviors: `spp_orig`, `spp_dev` with FDP-style controls, and `bop`. The selector asks two questions on every stable window: what does the local stream look like, and does the peer make the locally preferred choice risky? Our research question is whether these local and peer/shared signals can choose safer prefetch behavior than a single static policy. For the retained workload suite, the results show that these signals improve over a static `SPP_orig` baseline, while the Phase 2 mini-evaluation also reveals a limitation: the coordinator reduces but does not fully eliminate the `mcf + lbm` regression.

2. Related Work

Feedback Directed Prefetching (FDP) by Srinath et al. dynamically adjusts prefetch aggressiveness using feedback such as accuracy, timeliness, and pollution [1]. Our `spp_dev` expert contains an FDP-style controller, but the final design builds above it: it selects among experts and uses pair signals to avoid choices that are unsafe under shared contention.

The Signature Path Prefetcher (SPP), based on path-confidence lookahead prefetching, is our main SPP baseline [2]. It uses signatures and confidence to look ahead in access streams. Best-Offset Prefetching (BOP) searches for spatial offsets that work for the current stream [3]. In our design, these mechanisms are not only baselines but also expert behaviors: SPP represents the strong default lookahead prefetcher, the FDP-style variant represents a feedback-controlled SPP mode, and BOP represents an offset-oriented alternative for streams where it behaves differently. BWC differs by adding a selector and pair coordinator above the experts rather than replacing their address-generation logic. ChampSim is the trace-based simulation framework used to evaluate these designs [4]. OpenEvolve is an evolutionary coding framework with LLM-guided mutation and quality-diversity search [5]. We use it in a narrow, coordinator-only way so the evolved design remains interpretable.

3. Method

Implementation Overview

The main implementation is in `prefetcher/adaptive_selector/`. The Phase 2 retained best point is stored separately in `prefetcher/adaptive_selector_phase2_best/`. Table 1 maps the main selector stages to source-level artifacts.

Figure 1 shows the final design. Each demand access is pushed into a sliding window, and the window produces four local features. A local classifier proposes an expert. In two-core mode, shared snapshots record peer behavior and pressure.

Table 1: Source artifacts and implementation map.

Stage	Main function / artifact	Role
Feature extraction	`compute_window_features()`	Computes `page_growth`, `line_growth`, `small_delta_ratio`, and `rfo_share`.
Local selection	`classify_window()`	Produces the local `orig`, `FDP`, or `BOP` candidate.
Shared context	`refresh_shared_pressure()`	Records local pressure plus peer page/delta signatures and `peer_lbm_like`.
Pair coordination	`coordinate_candidate()`	Remaps or blocks risky pair choices before activation.
Configurations	`adaptive_selector` configs	Instantiate the selector in single-core and two-core ChampSim runs.

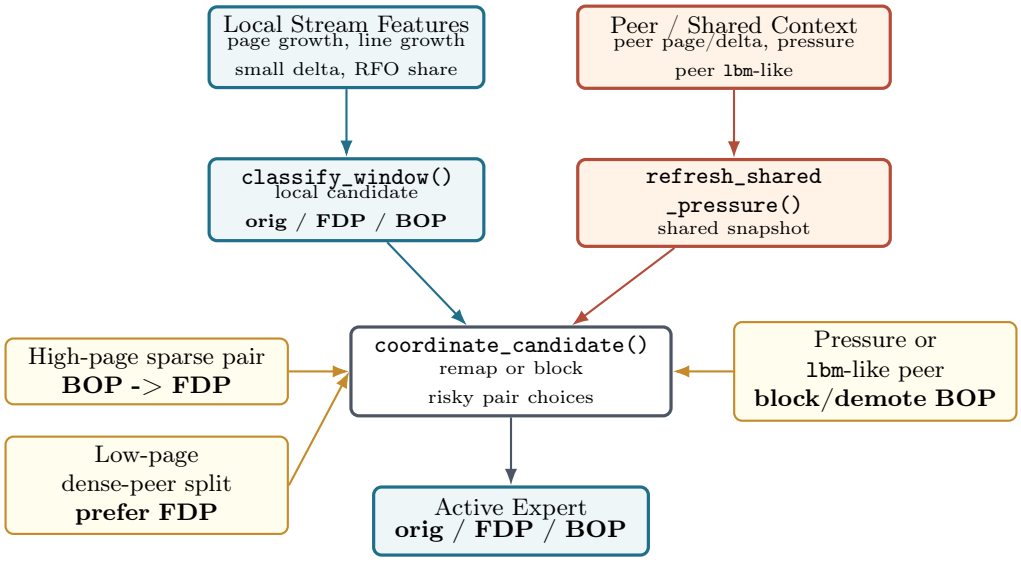

Figure 1: Simplified BWC Adaptive Selector pipeline, based on the poster flow. Local stream features select a candidate; peer/shared signals guard the final activation.

The coordinator then applies pair-aware rules before the candidate becomes the active expert. We use the name FDP for the FDP-style SPP expert in the selector; it is not a full reimplementation of the FDP paper, but an SPP variant with feedback-style controls intended to be safer under some traffic patterns. Overall, the simulated policy is relatively hardware-conscious: it relies on short counters, ratios, threshold comparisons, and a small shared state vector rather than expensive global history.

OpenEvolve Setup

The original proposal planned a parameter-only OpenEvolve search over an SPP controller. During development, the retained artifact shifted to expert selection because it gave clearer control over heterogeneous pair behavior. For Phase 2, OpenEvolve searched only the coordinator logic. The expert prefetchers stayed fixed. The best retained point changed several pair fallbacks from `orig` to FDP, including dense-peer split handling, pair-scope BOP demotion, and `lbm`-like peer protection. It also added two runtime guards for sparse/high-page-growth cores paired with `lbm`-like peers, and for `lbm`-like cores paired with high-page-growth peers.

For OpenEvolve, we constrained mutations to the pair-coordination decision path, primarily `coordinate_candidate()`, and kept the expert prefetchers and ChampSim interfaces fixed. The prompt emphasized preserving the Phase 1 selector behavior while improving fragile `lbm`-related pair cases. Each candidate was evaluated on a focused 2M-warmup plus 10M-simulation mini-suite containing single-core `lbm`, `bzip2 + lbm`, and `mcf + lbm`. The score combined mean gain with a worst-case guardrail so that a candidate had to improve pair behavior without introducing a new large regression. We retained the best candidate according to this focused score and report the broader final performance summary separately.

Evaluation Setup

4. Experimental Results

Phase 2 OpenEvolve best. The Phase 2 OpenEvolve numbers are from a short focused mini-evaluation used to compare Phase 1 and Phase 2 coordinator variants. The retained single-core and two-core tables that follow report the final evaluation ledger. Thus, Table 3 should be read as an optimization diagnostic, while Tables 5–7 are the final reported performance summary. Figure 2 visualizes the before/after gains.

The aggregate score improves from -6.2665 to -3.0105, while mean gain and worst-case gain improve by 0.1951 and 0.4747 percentage points, respectively.

Retained single-core results. The selector remains positive on every reported isolated workload, which was a necessary guardrail before trusting the two-core rules. Table 5 reports the raw IPC values, while Figure 3 shows the gain distribution.

Retained two-core results. The retained pair set is also positive across all listed heterogeneous and homogeneous mixes. The `lbm` pairs are the main stress cases because their local prefetch behavior can become harmful when another core competes for shared resources. The smaller but positive gains on `bzip2 + lbm` and `mcf + lbm` suggest that the coordinator is most

Table 2: Evaluation setup.

Item	Setting
Simulator	ChampSim trace-based simulation.
Primary baseline	`SPP_orig`; reported gains are relative to this baseline.
Selector configs	`configs/adaptive_selector_config.json` and `configs/adaptive_selector_2core.json`.
Experts	Original SPP, FDP-style SPP, and BOP.
Workloads	`astar`, `mcf`, `lbm`, and `bzip2`.
Metrics	Single-core IPC and two-core weighted speedup.
Weighted-speedup definition	Sum of each core's shared-run IPC normalized to its corresponding single-core IPC; gains are relative to the `SPP_orig` pair baseline.
Final result source	Retained result ledger and associated scripts; the Phase 2 focused comparison uses the explicit short setting below.
Phase 2 run length	2M warmup plus 10M simulation for the focused OpenEvolve comparison.

Table 3: Phase 2 OpenEvolve best point, copied from the retained result ledger.

Case	SPP base	Phase 1	P1 gain	Phase 2	P2 gain	Delta
`lbm` IPC	0.866441	0.869666	+0.3722%	0.869666	+0.3722%	+0.0000 pp
`bzip2 + lbm` WS	1.792235	1.791723	-0.0286%	1.793705	+0.0820%	+0.1106 pp
`mcf + lbm` WS	1.256300	1.243540	-1.0157%	1.249503	-0.5410%	+0.4747 pp

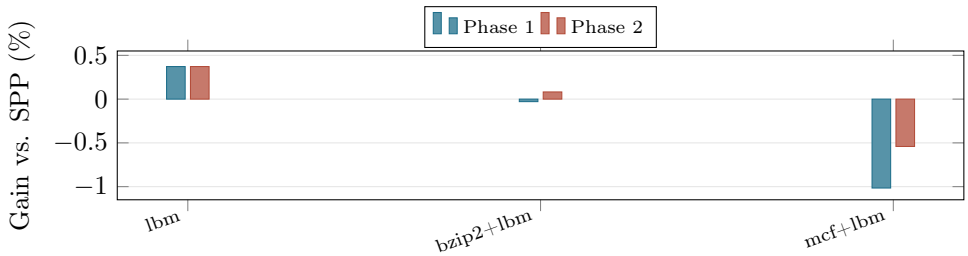

Figure 2: Phase 2 OpenEvolve before/after gains. OpenEvolve leaves single-core `lbm` unchanged, fixes the small `bzip2 + lbm` regression, and reduces the `mcf + lbm` loss.

Table 4: Phase 2 aggregate metrics.

Metric	Phase 1 source	Phase 2 best	Delta
`combined_score`	-6.2665	-3.0105	+3.2559
`mean_gain_pct`	-0.2240%	-0.0289%	+0.1951 pp
`min_gain_pct`	-1.0157%	-0.5410%	+0.4747 pp

Table 5: Single-core IPC results.

Workload	Baseline IPC	Current IPC	Gain
`astar`	0.116244	0.117697	+1.2498%
`mcf`	0.093047	0.095530	+2.6685%
`lbm`	0.870554	0.887491	+1.9456%
`bzip2`	2.132624	2.150899	+0.8569%

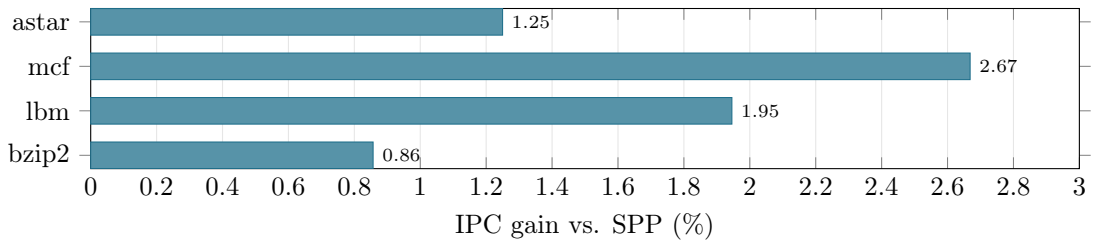

Figure 3: Single-core IPC gain. The selector preserves isolated behavior while enabling pair-aware control.

useful as a guardrail: it reduces harmful pair interactions while preserving positive final weighted speedup.

Table 6: Two-core heterogeneous weighted-speedup results.

Pair	Baseline WS	Current WS	Gain
`mcf + astar`	1.777787	1.824991	+2.6552%
`astar + bzip2`	1.871024	1.899359	+1.5144%
`bzip2 + mcf`	1.868884	1.894343	+1.3623%
`astar + lbm`	1.356202	1.375121	+1.3950%
`bzip2 + lbm`	1.938392	1.949615	+0.5790%
`mcf + lbm`	1.282647	1.297473	+1.1559%

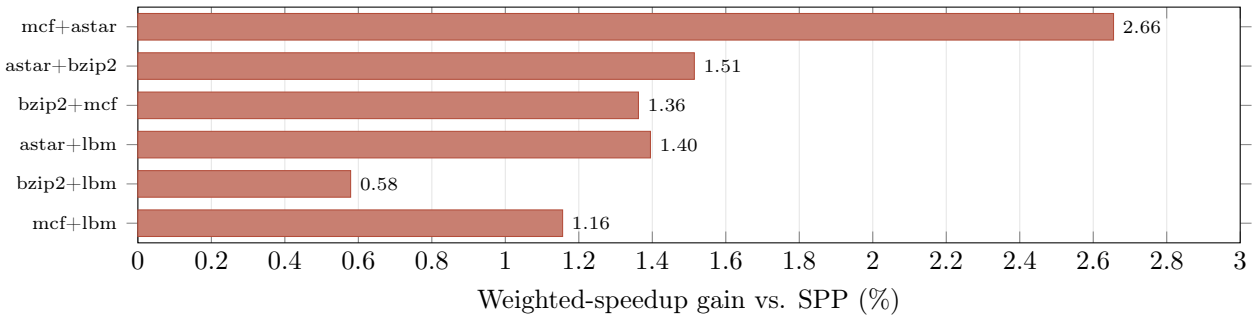

Figure 4: Heterogeneous two-core gains. The `lbm` pairs are the most diagnostic cases for peer-aware protection.

Table 7: Two-core homogeneous weighted-speedup results.

Pair	Baseline WS	Current WS	Gain
`astar + astar`	1.598755	1.666130	+4.2142%
`mcf + mcf`	1.703222	1.733555	+1.7809%
`lbm + lbm`	1.054659	1.056457	+0.1705%
`bzip2 + bzip2`	2.016283	2.017322	+0.0515%

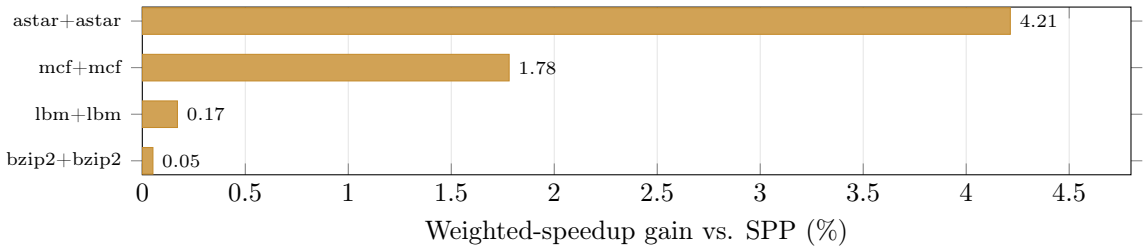

Figure 5: Homogeneous two-core gains. The selector's largest retained win is `astar + astar`; the `bzip2 + bzip2` result is close to neutral and should be read as preservation rather than a large speedup.

Experimental process. The process was iterative. Intermediate rules were kept only when they improved a target pair without breaking previously retained single-core and pair behavior. Table 8 summarizes the commit progression behind that narrow final design.

Table 8: Adaptive-selector commit progression.

Commit	Stage	Contribution
`40deaeb4`	Sliding-window baseline	Introduced the adaptive selector and retained first single-core summaries.
`2535efab`	Rule-2 selector	Reproduced direct expert matrices and tightened selector behavior.
`051aaca8`	Pair-safe selector	Added shared coordination and retained `mcf + lbm` pair artifacts.
`bc78d4bf`	Generalized pair state	Extended pair validation to `astar + lbm`, `bzip2 + lbm`, and `mcf + lbm`.
`27258b89`	Final state retention	Consolidated the final adaptive-selector behavior in root results.
`7a743462`	Phase 1 final	Finalized the retained Phase 1 selector source.
`e65a5b4e`	Phase 2 OpenEvolve best	Recorded the coordinator-only OpenEvolve best point.

This is why the final design is intentionally narrow: it preserves the useful local expert behavior and adds only enough peer-aware coordination to avoid the observed harmful interactions.

5. Goals and Next Steps

The original proposal goal was an OpenEvolve-optimized bandwidth-aware SPP controller. We partially met that goal, but the final artifact shifted to a stronger adaptive-selector formulation. Instead of tuning a single SPP controller, we select among SPP, FDP-style SPP, and BOP. This better matches the observed problem: different workloads and pairs need different expert styles, not only different throttling thresholds. Although this shift changed the mechanism, it preserved the proposal's larger goal: using automated search to improve prefetch control under shared-resource pressure.

Future work should evaluate more held-out SPEC mixes, run longer and repeated simulations, validate four-core behavior, and use OpenEvolve over a broader but still constrained coordinator rule space. The most useful next analysis is an ablation suite that disables one pair rule at a time and reports prefetch traffic, cache pollution, and queue pressure alongside IPC and weighted speedup.

6. Collaboration

Zhaofeng Luo focused on experiment infrastructure, running ChampSim evaluations, collecting IPC and weighted-speedup summaries, and preparing the final result tables and poster figures. He also led the final report and poster writing, helped analyze the single-core and two-core results, and validated the retained result ledger.

Kijun Shin focused on the adaptive-selector implementation, pair-coordination rules, and OpenEvolve-guided Phase 2

refinement. He also helped debug the coordinator behavior and interpret the evolved changes to `coordinate_candidate()`. Both members jointly designed the BWC selector idea, studied related prefetching work, discussed evaluation methodology, reviewed intermediate results, and refined the final project narrative.

7. Conclusion

The BWC Adaptive Selector demonstrates that prefetch control should not be purely local. The selector uses local stream features to choose an expert and peer/shared signals to decide whether that choice is safe for the memory system. The retained results show positive gains for every listed single-core workload and two-core pair in our reported suite. The Phase 2 OpenEvolve best point further shows that narrow, coordinator-only evolution can improve difficult `lbm` interactions without changing expert predictor internals. The central lesson is simple: the right prefetcher for one core is not always the right prefetcher for the system.

References

[1] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt. Feedback Directed Prefetching: Improving the Performance and Bandwidth-Efficiency of Hardware Prefetchers. HPCA, 2007. https://hps.ece.utexas.edu/pub/srinath_hpca07.pdf

[2] J. Kim, S. H. Pugsley, P. V. Gratz, A. L. N. Reddy, C. Wilkerson, and Z. Chishti. Path Confidence Based Lookahead Prefetching. MICRO, 2016. https://dblp.org/rec/conf/micro/KimPGRWC16.html

[3] P. Michaud. Best-Offset Hardware Prefetching. HPCA, 2016. https://core.ac.uk/outputs/48160133/

[4] N. Gober, G. Chacon, L. Wang, P. V. Gratz, D. A. Jimenez, E. Teran, S. H. Pugsley, and J. Kim. The Championship Simulator: Architectural Simulation for Education and Competition. arXiv:2210.14324, 2022. https://arxiv.org/abs/2210.14324

[5] OpenEvolve project repository. https://github.com/algorithmicsuperintelligence/openevolve Accessed April 2026.